CS 255: Computer Architecture and Machine Level Programming

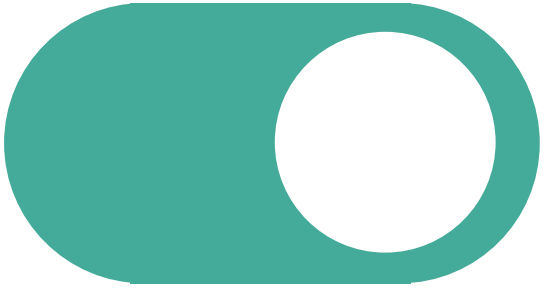
Prof. Martha Barker

Memory

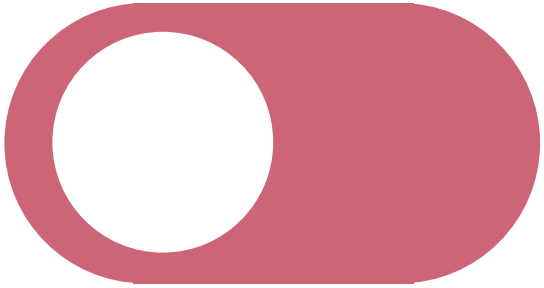
Memory Device

A **memory device** stores data over time.

A **switch stores** two states: off and on.



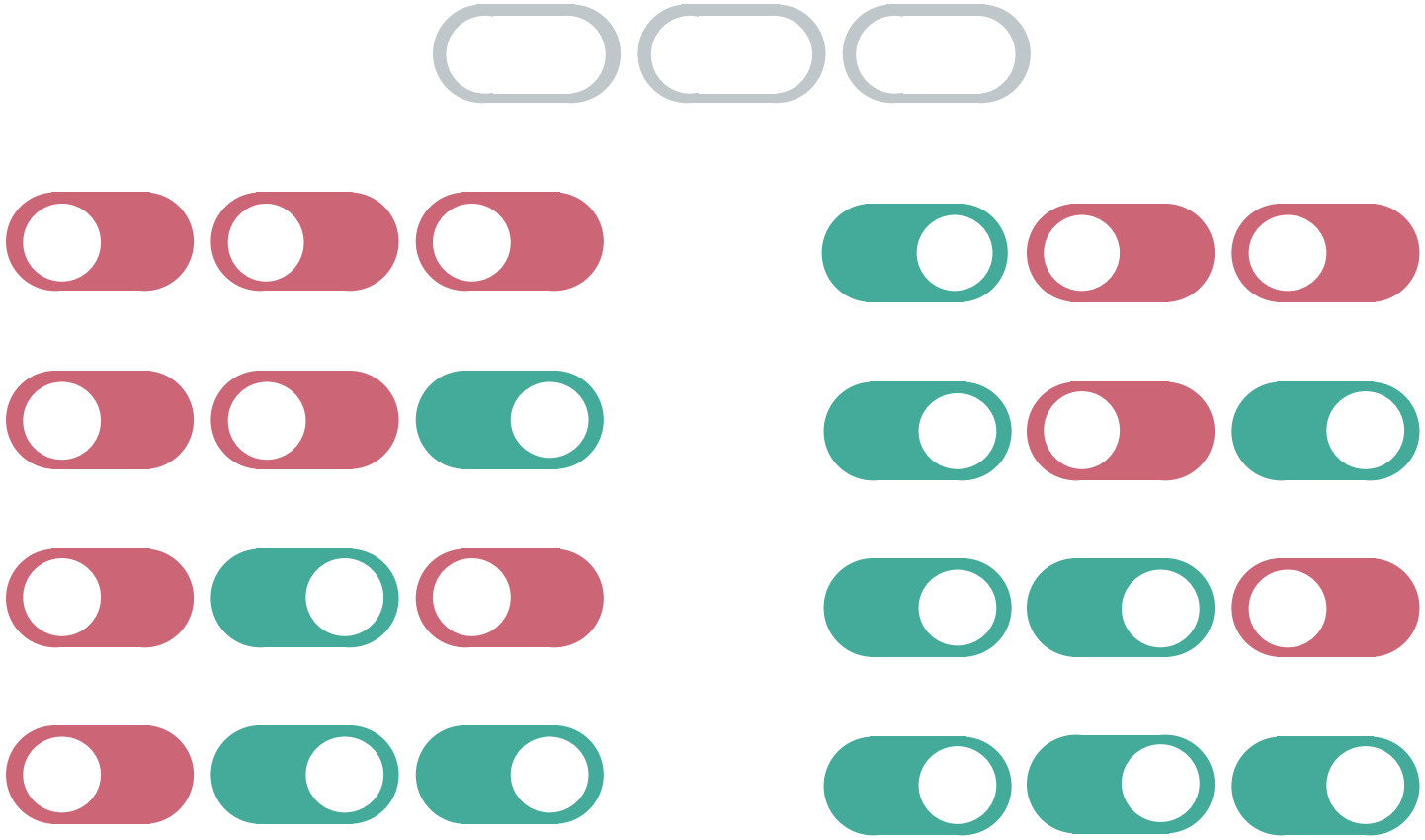
True
ON
1



False
OFF
0

Combine Devices to Store Larger Range

n switches can store 2^n values



Vocabulary

Bit: a single binary digit

Byte: group of 8 bits treated as a unit

Word: natural unit of data for a processor (e.g. 32 or 64 bits)

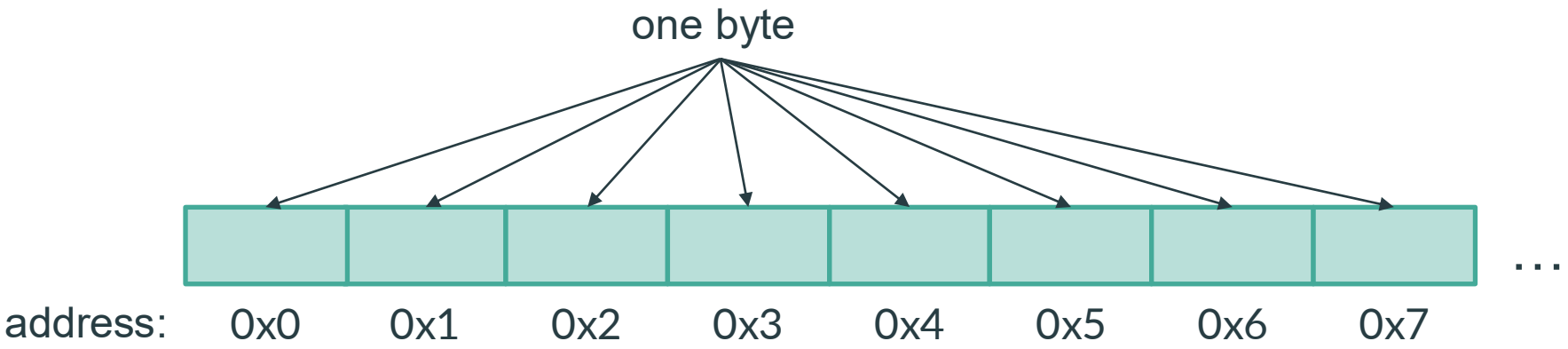
Most Significant Bit (MSB): bit with the largest value, typically on the left

Least Significant Bit (LSB): bit with the smallest value, typically on the right

Memory Layout

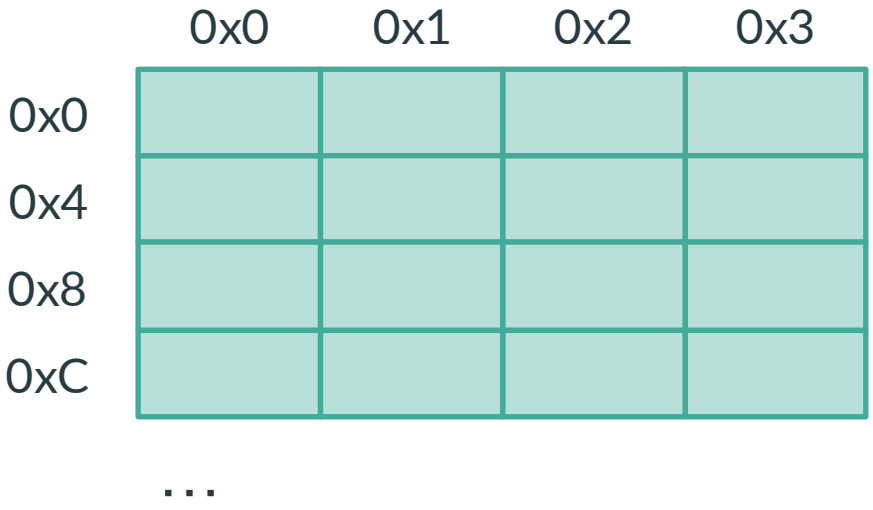
Memory is an Array of Bytes

Each element is one byte (eight bits) which is identified by a unique address.



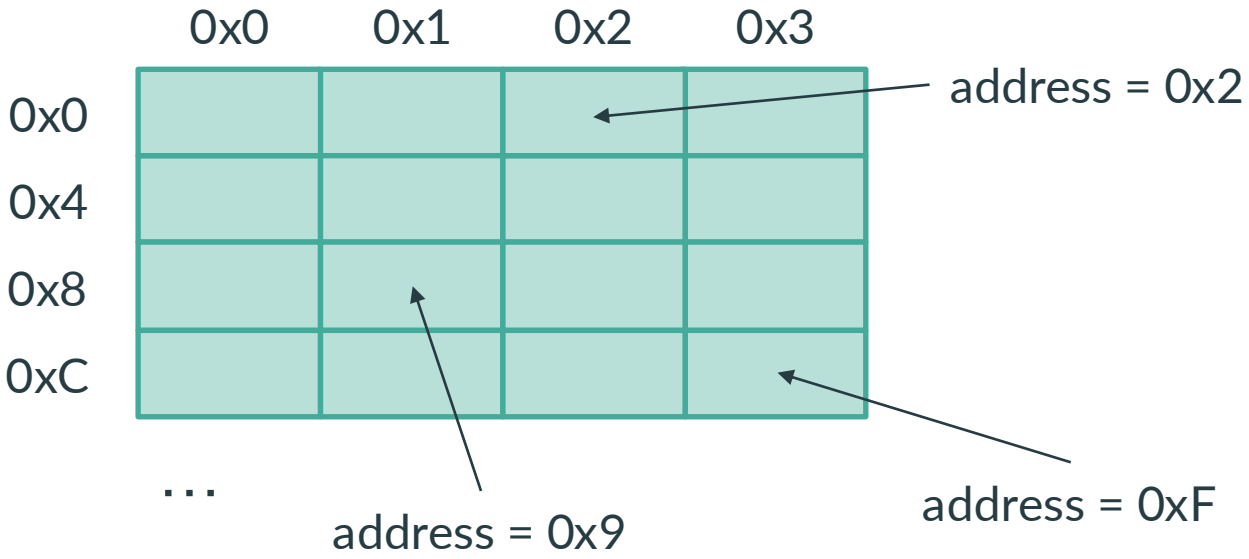
A More Compact Notation

We can organize our array as a grid to make it more compact.



A More Compact Notation

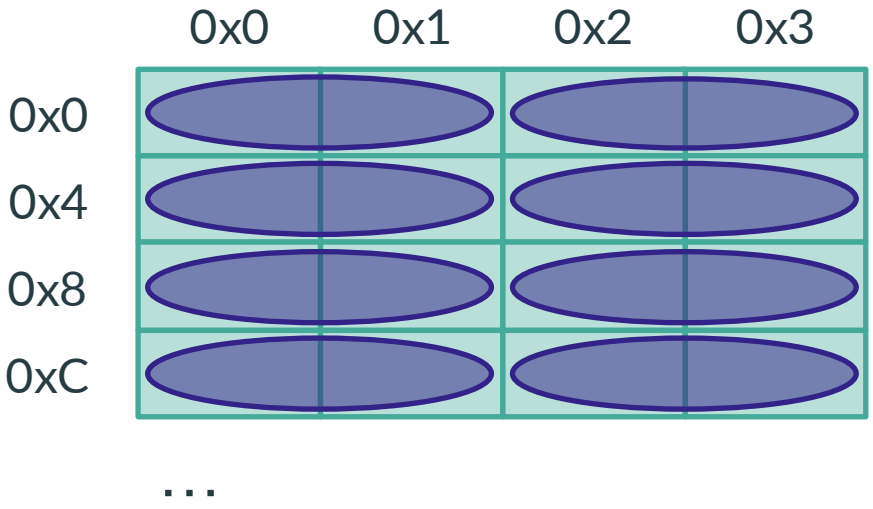
We can organize our array as a grid to make it more compact.



address = row address + column address

Combining Two Adjacent Bytes

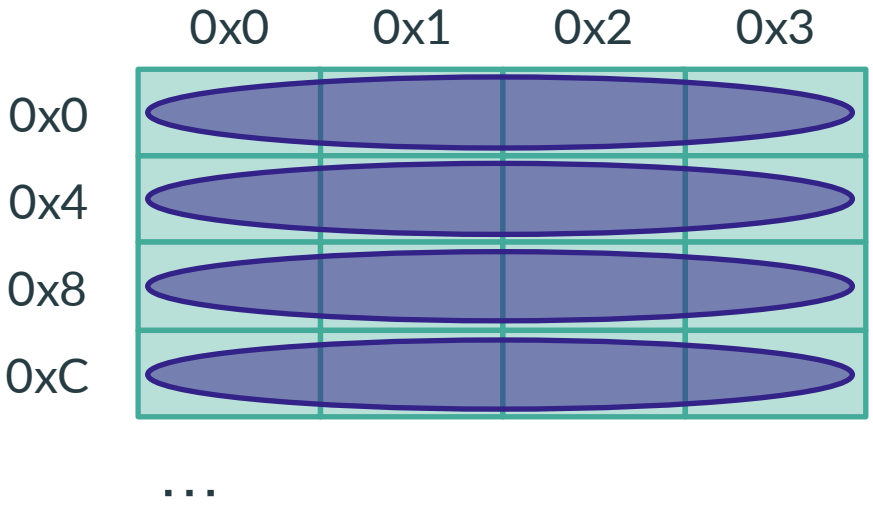
Two adjacent bytes can be combined to store 2^{16} bit patterns.



★ first byte must have an even address

Combining Four Adjacent Bytes

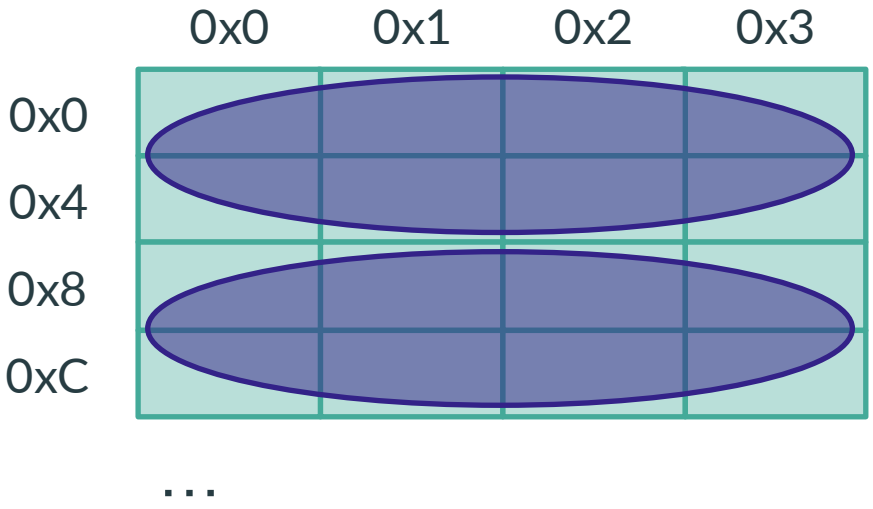
Four adjacent bytes can be combined to store 2^{32} bit patterns.



★ first byte must have an address divisible by 4

Combining Eight Adjacent Bytes

Eight adjacent bytes can be combined to store 2^{64} bit patterns.



★ first byte must have an address divisible by 8

Storing an Integer

int x = 123456; // 123456 = 0x1E240

	0x0	0x1	0x2	0x3
0x0	0x00	0x01	0xE2	0x40
0x4				
0x8				
0xC				

an integer is 4 bytes, so it can be stored at any address divisible by 4

Storing an Integer Array

```
int arr[3] = {10, 20, 30};
```

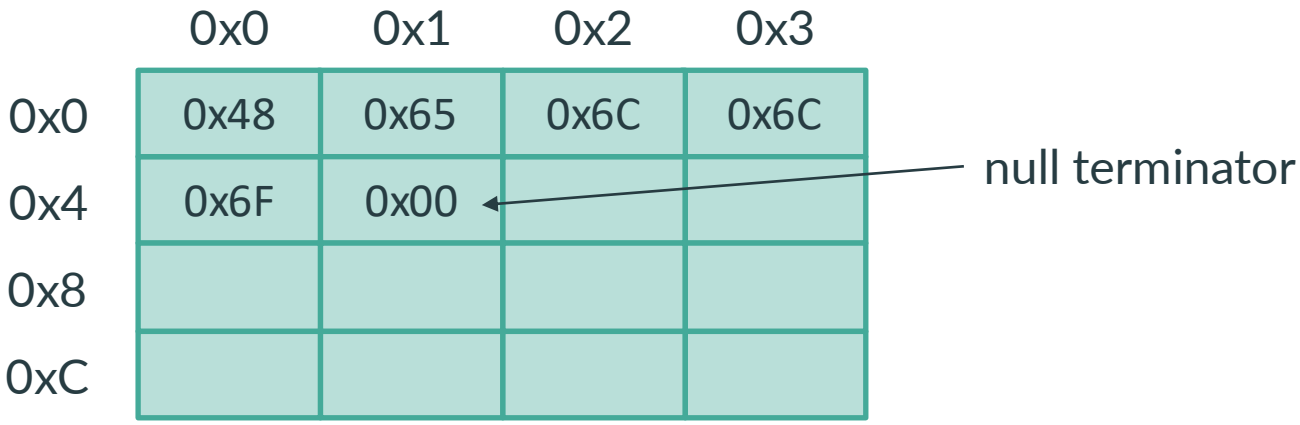
	0x0	0x1	0x2	0x3
0x0	0x00	0x00	0x00	0x0A
0x4	0x00	0x00	0x00	0x14
0x8	0x00	0x00	0x00	0x1E
0xC				

each int is still stored at an address divisible by 4

arrays are contiguous in memory, so elements are stored one after the other

Storing a String

```
char str[6] = "Hello";
```



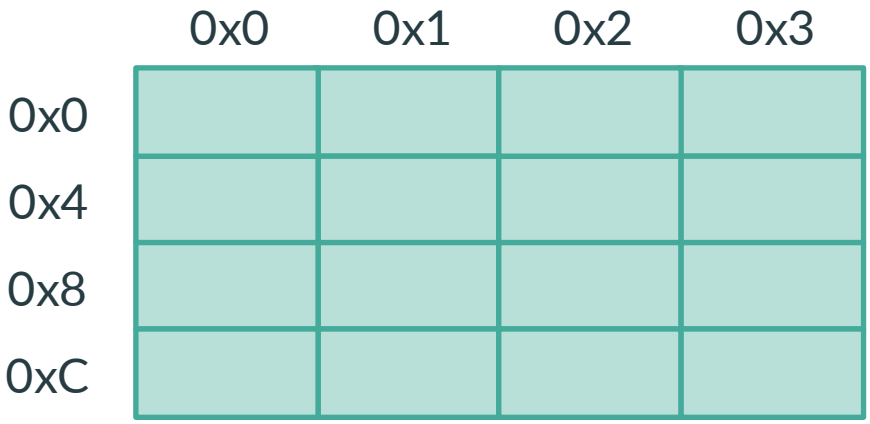
each char is stored at an address divisible by 1

arrays are contiguous in memory, so elements are stored one after the other

Storing a Struct

Structs are aligned to the width of their widest field, and each field is aligned to its size.

```
struct char_char_int{
        char c1;
        char c2;
        int i;
};
```



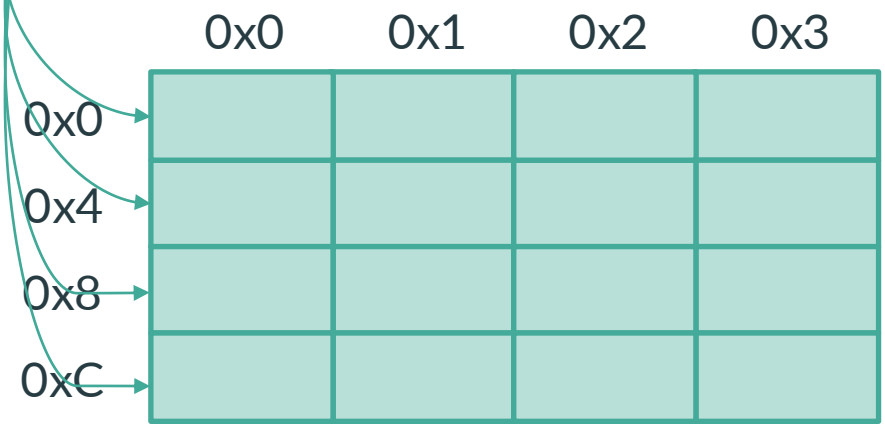
Storing a Struct

Structs are aligned to the width of their widest field, and each field is aligned to its size.

widest field is an int (4 bytes)

```
struct char_char_int{
        char c1;
        char c2;
        int i;
};
```

so struct must be stored at an address divisible by 4



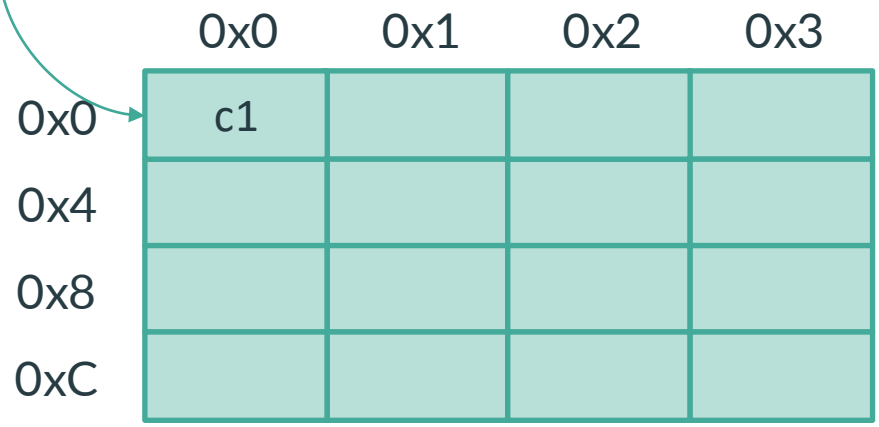
Storing a Struct

Structs are aligned to the width of their widest field, and each field is aligned to its size.

the first field is 1 byte

```
struct char_char_int{
        char c1;
        char c2;
        int i;
};
```

but it's the first, so it will be stored at an address divisible by 4



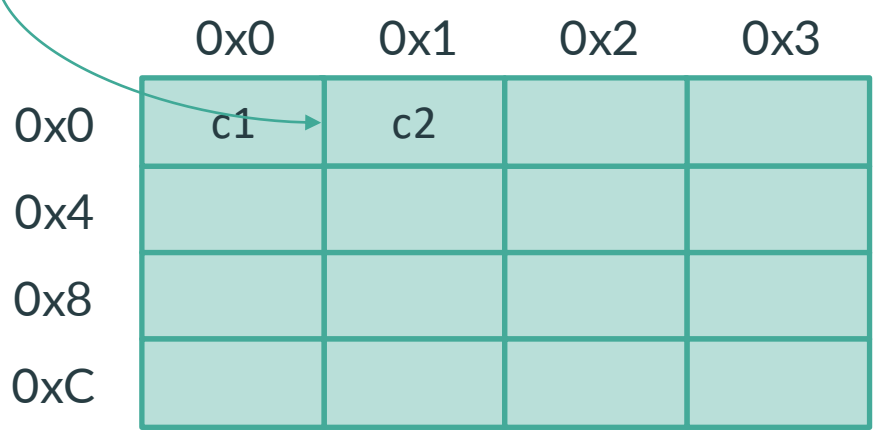
Storing a Struct

Structs are aligned to the width of their widest field, and each field is aligned to its size.

the next field is 1 byte

```
struct char_char_int{
        char c1;
        char c2;
        int i;
};
```

so it will be stored at the next address divisible by 1



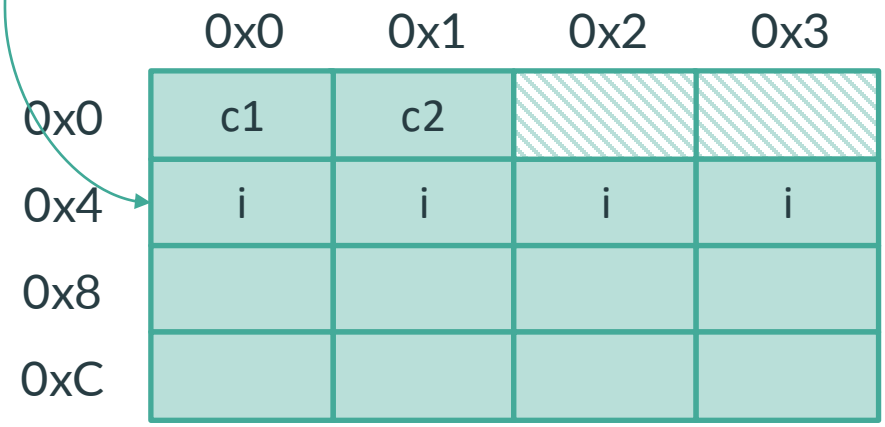
Storing a Struct

Structs are aligned to the width of their widest field, and each field is aligned to its size.

the next field is 4 bytes

```
struct char_char_int{
        char c1;
        char c2;
        int i;
};
```

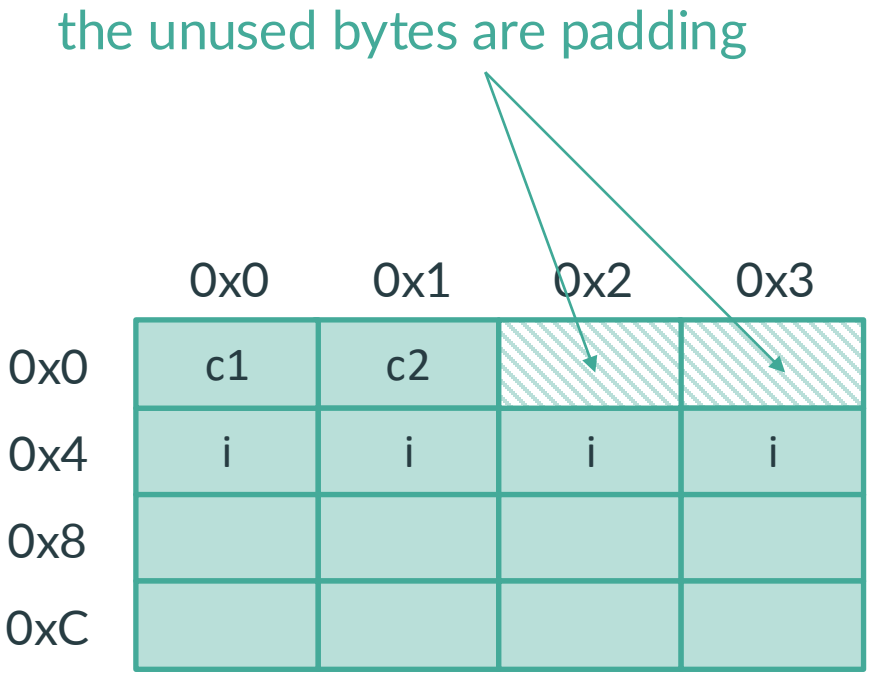
so it will be stored at the next address divisible by 4



Storing a Struct

Structs are aligned to the width of their widest field, and each field is aligned to its size.

```
struct char_char_int{
        char c1;
        char c2;
        int i;
};
```

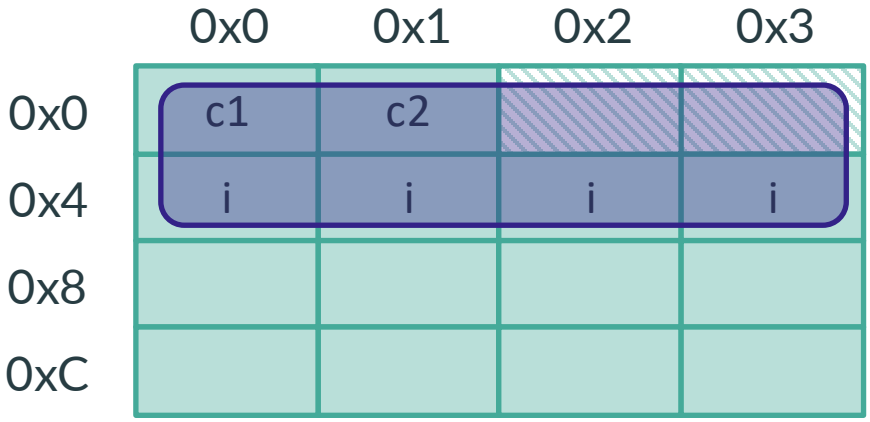


Storing a Struct

Structs are aligned to the width of their widest field, and each field is aligned to its size.

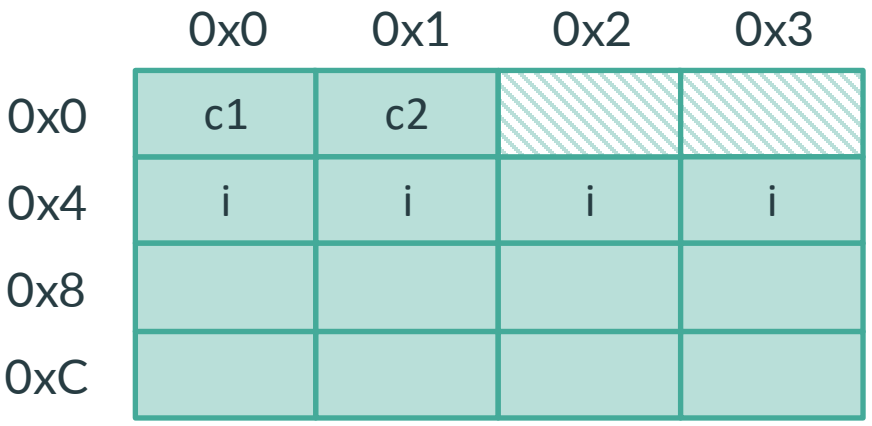
total size is 8 bytes

```
struct char_char_int{
        char c1;
        char c2;
        int i;
};
```

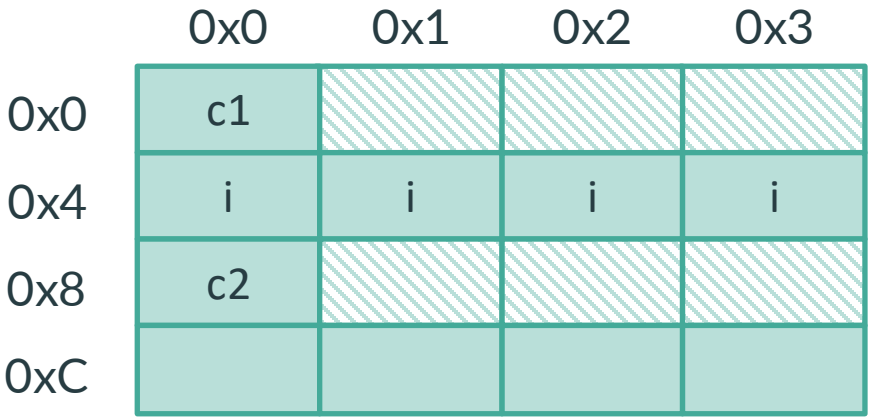


Struct Layout Matters

```
struct char_char_int{
        char c1;
        char c2;
        int i;
};
```




```
struct char_int_char{
        char c1;
        int i;
        char c2;
};
```

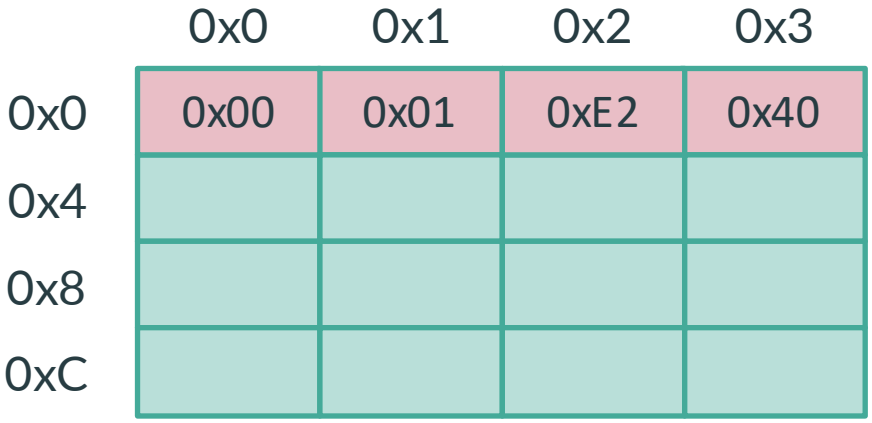


Alignment Makes Memory Accesses Fast

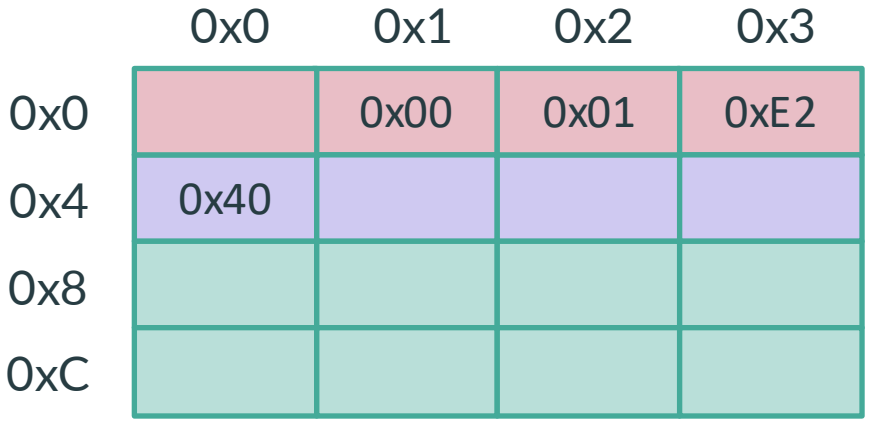
CPUs read and write memory one word at a time:

- 32-bit CPU reads/writes 4 bytes per access
- 64-bit CPU reads/writes 8 bytes per access

If data is aligned, reading an int takes one memory access:



If it isn't, reading an int takes two memory accesses and a merge:

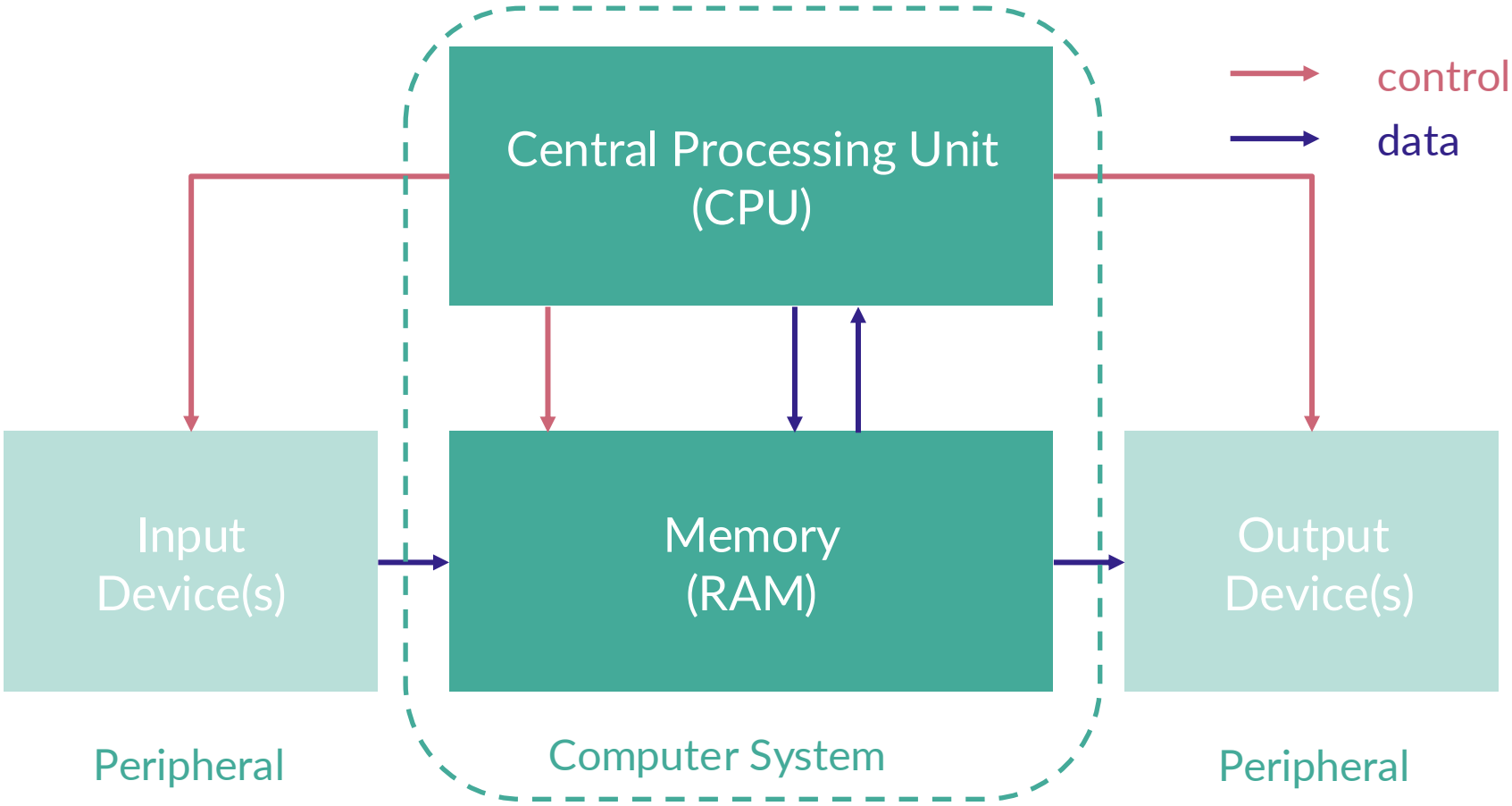


Exercise: Memory Layout

Data Transfer

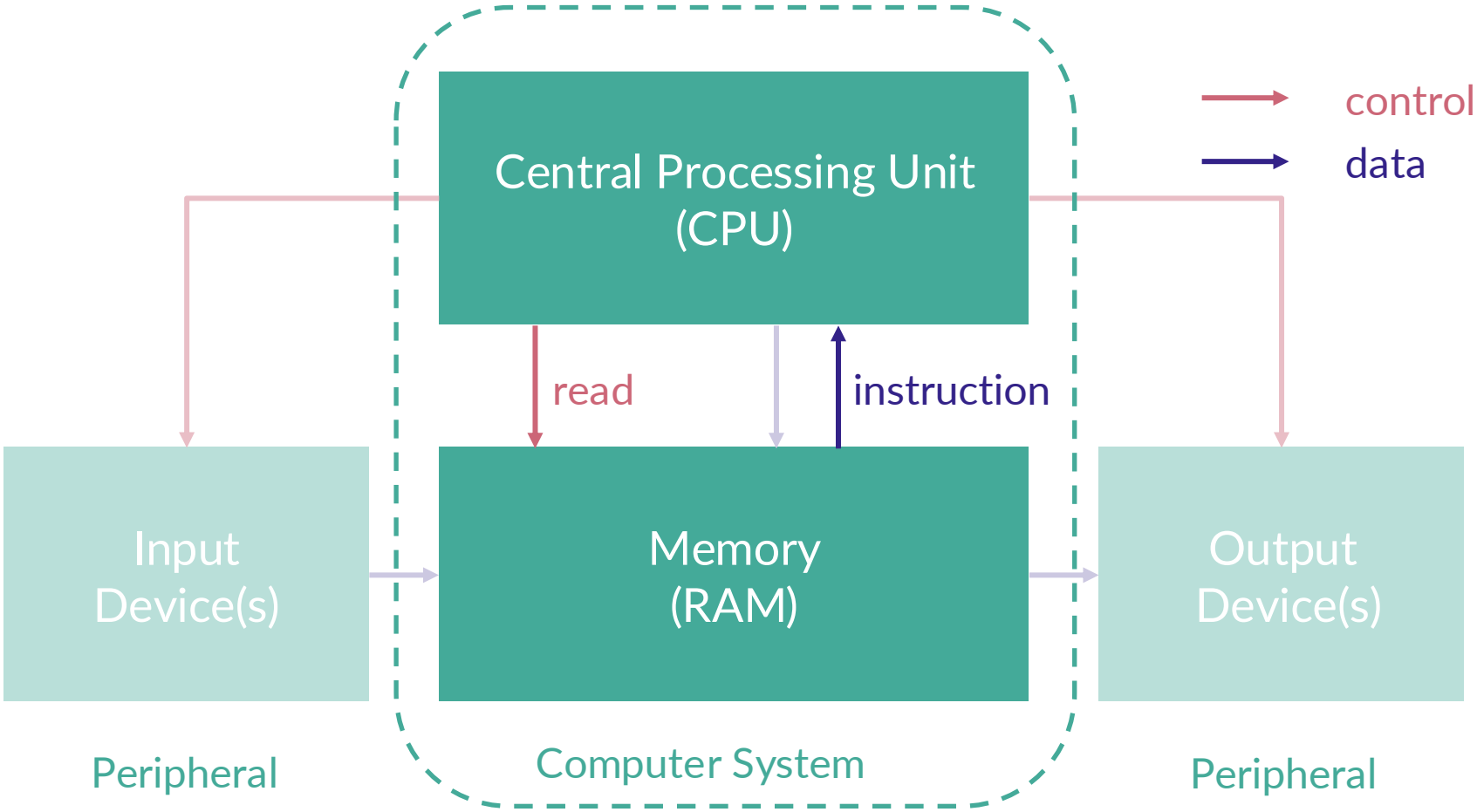
All Data Goes Through Memory

Memory is a storage array that stores (1) computer program instructions and (2) data used by the program.



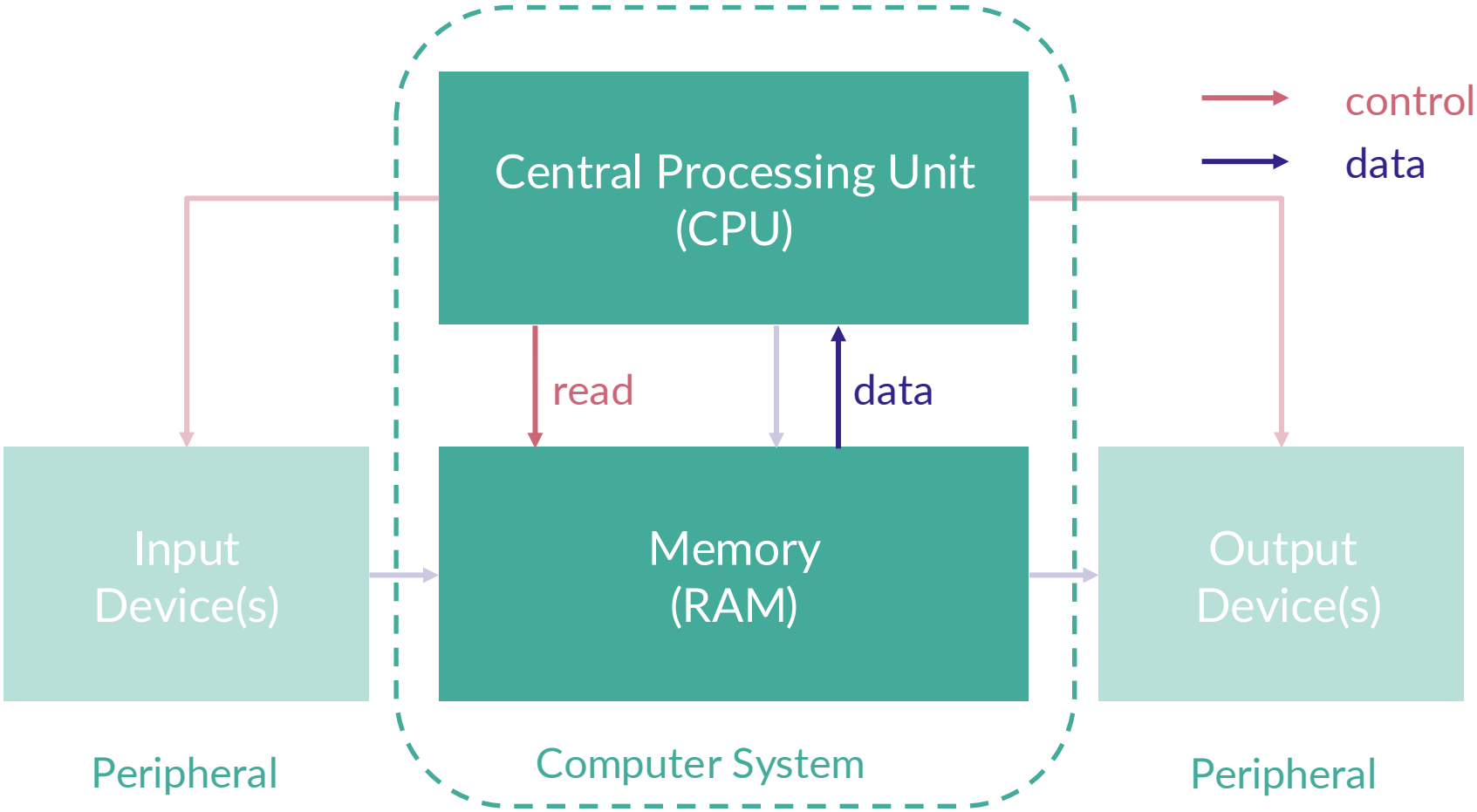
The CPU Reads Instructions

The memory stores all program instructions.



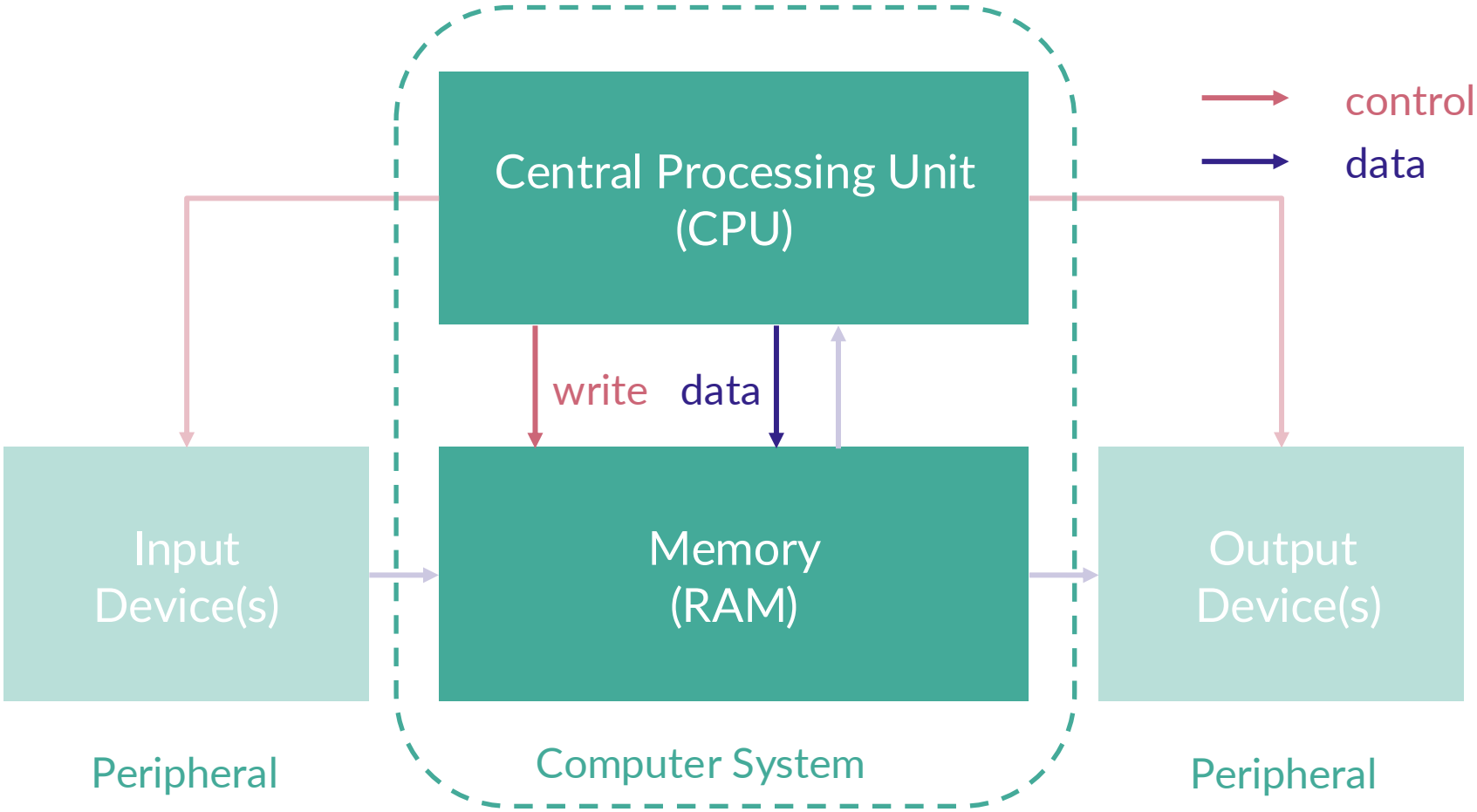
The CPU Also Reads Data

The memory stores all variables used by program instructions.



The CPU Writes Data

Memory can be updated when the CPU changes variable values.



Wider Data Bus Allows Faster Transfer



2 lane highway →
1 car in each direction



6 lane highway →
3 cars in each direction

Wider Data Bus Is More Expensive



2 lane highway →
1 car in each direction

Less space and materials to build
Fewer cars means less gas



6 lane highway →
3 cars in each direction

More space and materials to build
More cars means more gas

Width of Data Bus

The *width* of the data bus is the number of wires, each of which transfers one bit per read/write operation.

Data Bus Width	Data Transfer Per Operation
8 bits	1 byte
16 bits	2 bytes
32 bits	4 bytes
64 bits	8 bytes

★ Data bus width is always a power of 2

Width of Address Bus

The *width* of the address bus determines the number of unique memory addresses, or the maximum size of memory that can be accessed

Address Bus Width	Maximum Memory Size
8 bits	256 bytes
16 bits	64 kilobytes
32 bits	4 gigabytes
64 bits	16 exabytes

★ Address bus width can be any integer